

# Software Design Description (SDD) for SENA Network

Document Version: 1.1.0

## 1. Introduction

### 1.1 Purpose

This Software Design Description (SDD) provides a detailed technical design for the SENA Network, a Layer 2 (L2) modular infrastructure. It elaborates on the architectural components, module designs, data structures, and algorithms necessary to implement the requirements outlined in the Software Requirements Specification (SRS). This document is intended for developers, architects, and technical stakeholders involved in the construction and maintenance of the SENA Network.

### 1.2 Scope

This SDD covers the design of the core SENA L2 protocol, including its sequencer, state management, and integration with Aptos L1. It details the design of the Keyless Authentication Engine, Social Connect Identity Index, Multi-Asset Gas Paymaster Engine, Optimistic L2 Governance Layer, and the Fraud Proof and Dispute Resolution System, with a particular focus on the Rust implementation using Sovereign SDK and RocksDB, and the integration of the "Aptos-Level Secure" model.

### 1.3 Definitions, Acronyms, and Abbreviations

Refer to Section 1.3 of the Software Requirements Specification (SRS) for a comprehensive list of definitions, acronyms, and abbreviations.

### 1.4 References

- [1] Software Requirements Specification (SRS) for SENA Network
- [2] SENA Network: Layer 2 Modular Infrastructure Documentation (Stakeholder Document)
- [3] Sovereign SDK Documentation. Sovereign Labs. Available at: <https://docs.sovereign.xyz/>
- [4] RocksDB. GitHub. Available at: <https://github.com/facebook/rocksdb>
- [5] Aptos Keyless Accounts. Aptos Docs. Available at: <https://aptos.dev/concepts/accounts/#keyless->

accounts

- [6] NOMT: Nearly-Optimal Merkle Trie. GitHub. Available at: <https://github.com/aptos-labs/aptos-core/tree/main/storage/aptosdb/src/schema/nomt>

## 2. System Architecture

### 2.1 High-Level Architecture

SENA Network operates as a Sovereign SDK-based rollup, with its core logic implemented in Rust. It functions as a single-sequencer L2, periodically committing its state to the Aptos L1 for security and finality. The architecture is designed for modularity, performance, and resource efficiency.

The control flow divides into three planes:

- **The execution plane** — the sequencer and the Sovereign SDK runtime, which order and execute transactions against RocksDB-backed state, producing soft confirmations within seconds.
- **The settlement plane** — the Aptos L1 Move contracts, which record bonded assertions about the L2 state, hold bridged assets, and adjudicate disputes. Nothing on this plane trusts the execution plane.
- **The verification plane** — independent verifier nodes, which re-execute the published batch data and challenge any assertion that disagrees with their own result. This plane is permissionless and is what makes the settlement plane's optimism safe.

A single participant may occupy more than one plane, but the security argument depends on the verification plane containing at least one honest party who is not the sequencer.

### 2.2 Component Breakdown

#### 2.2.1 SENA L2 Sequencer

- **Role:** The central processing unit of the L2. It receives transactions from client applications, orders them, executes them against the Sovereign SDK runtime, and periodically commits state roots to Aptos L1.
- **Technology:** Rust application, leveraging Sovereign SDK components.
- **Key Responsibilities:**
  - Transaction reception and mempool management.
  - Transaction ordering and block production.
  - Interaction with the Sovereign SDK runtime for state transitions.
  - Publication of batch transaction data for data availability.

- Generation and submission of bonded assertions to Aptos L1.
- Retention of execution checkpoints sufficient to defend its assertions during the challenge window.
- Integration with the Aptos Signature Verification Module for ZKP validation.
- Servicing forced-inclusion transactions drawn from the L1 inbox.

### 2.2.2 Sovereign SDK Runtime

- **Role:** Provides the execution environment for SENA's custom state transition logic. It defines the modules (Keyless Auth, Social Connect, Gas Paymaster, Governance) and their interactions.
- **Technology:** Rust modules, built using the Sovereign SDK framework.
- **Key Responsibilities:**
  - Managing the L2 state, including accounts, balances, and module-specific data.
  - Executing transaction logic according to the defined state transition function.
  - Interfacing with RocksDB for persistent storage.
  - Providing APIs for module interactions.
  - Maintaining strict determinism, as required by REQ-FRAUD-031.

### 2.2.3 RocksDB + NOMT Storage Layer

- **Role:** The persistent storage solution for the L2 state. RocksDB provides a high-performance key-value store, while NOMT ensures efficient Merkle tree operations for state proofs.
- **Technology:** RocksDB (C++ library with Rust bindings), NOMT (Rust implementation).
- **Key Responsibilities:**
  - Storing all L2 state data (accounts, balances, module data).
  - Maintaining a Merklized representation of the state for efficient proof generation.
  - Providing fast read and write access for the Sovereign SDK runtime.
  - Producing the inclusion proofs that One-Step Proofs carry to L1.

### 2.2.4 Aptos Signature Verification Module

- **Role:** A dedicated module within the L2 sequencer responsible for verifying Zero-Knowledge Proofs (ZKPs) for keyless authentication.
- **Technology:** Rust implementation, importing Aptos's open-source signature verification code.
- **Key Responsibilities:**
  - Receiving pre-computed ZK proofs from client applications.
  - Performing lightweight cryptographic verification of ZK proofs.
  - Validating the link between Web2 JWTs and ephemeral public keys.

### 2.2.5 Aptos L1 Move Contract Suite

- **Role:** The smart contracts deployed on the Aptos L1 blockchain that serve as the security anchor for SENA. Originally a single contract, the suite is decomposed into four cooperating modules so that dispute logic can be audited and upgraded independently of custody logic.
- **Technology:** Aptos Move language.

| Contract                      | Responsibility                                                                                                                                    |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sena::assertions</code> | Records the assertion chain, holds proposer bonds, tracks Pending / Challenged / Finalized / Rejected status, and enforces the challenge window.  |
| <code>sena::disputes</code>   | Runs the bisection game: accepts challenges, holds challenger bonds, records each move, enforces per-move timeouts and chess-clock budgets.       |
| <code>sena::osp</code>        | The one-step verifier. Executes a single isolated instruction of the STF and verifies NOMT inclusion proofs. Contains no privileged entry points. |
| <code>sena::bridge</code>     | Custodies bridged assets, honours withdrawals against finalized assertions only, and exposes the forced-inclusion inbox.                          |

- **Key Responsibilities:**
  - Receiving and storing L2 assertions, including state roots and batch data commitments.
  - Enforcing bonding, challenge windows, and finalization.
  - Adjudicating disputes to completion without a trusted third party.
  - Providing a mechanism for users to withdraw funds directly from L1 using L2 state proofs.
  - Managing the L1 bridge for asset transfers between L1 and L2.

### 2.2.6 Aptos Decentralized Prover Network

- **Role:** An external, decentralized network responsible for generating computationally intensive Zero-Knowledge Proofs (ZKPs) required for keyless authentication.
- **Technology:** External service, provided by Aptos.
- **Key Responsibilities:**
  - Receiving requests for ZKP generation from client applications.
  - Computing ZK circuits based on Web2 JWTs and ephemeral public keys.
  - Returning pre-computed ZK proofs to client applications.

## 2.2.7 Verifier Node

- **Role:** An independent node that re-executes the L2 from published batch data and challenges any assertion that does not match its own computed post-state root. The verifier is the active ingredient in the optimistic security model; an optimistic rollup with no verifiers is a custodial database with extra steps.
- **Technology:** The same Rust binary as the sequencer, run in `--mode verifier`. Sharing the binary is deliberate: it guarantees the verifier's execution semantics cannot drift from the sequencer's (NFR-MAINT-004).
- **Key Responsibilities:**
  - Following `sena::assertions` for newly posted assertions.
  - Retrieving the referenced batch data and re-executing it against its local state.
  - Comparing the computed post-state root against the asserted one.
  - Opening a challenge on divergence, and playing bisection to completion unattended.
  - Retaining execution checkpoints so it can produce a One-Step Proof on demand.
  - Raising data availability challenges when batch data cannot be retrieved.

## 2.2.8 Dispute Engine

- **Role:** The L2-side counterpart to `sena::disputes`. Drives the bisection protocol on behalf of whichever role the node is playing — defending an assertion as sequencer, or prosecuting one as challenger.
- **Technology:** Rust module within the SENA node.
- **Key Responsibilities:**
  - Replaying a disputed batch to reconstruct the execution trace.
  - Computing intermediate state commitments at the trace offsets bisection requires.
  - Selecting the correct half at each bisection round and submitting the move to L1 within the move timeout.
  - Assembling the final One-Step Proof, including NOMT inclusion proofs for every state element the disputed step touches.
  - Monitoring its own clock budget and escalating L1 gas price as a deadline approaches.

# 3. Module Design

## 3.1 Protocol-Enforced Keyless Authentication Engine

- **Design Principles:** Security, privacy, user-friendliness, and off-chain computation.
- **Components:**
  - **Client-Side Key Pair Generator:** Generates ephemeral key pairs upon user login via Web2

providers.

- **ZKP Request Handler:** Client-side logic to request ZK proofs from the Aptos Decentralized Prover Network.
- **Aptos Signature Verification Module (L2 Node):** As described in Section 2.2.4, this module performs the on-chain verification of ZK proofs.
- **Account Derivation Logic:** Implements the deterministic L2 address derivation using \$aud, \$sub, and a pepper value.
- **Flow:**
  - User logs in via Google/Apple on a client application.
  - Client generates an ephemeral key pair.
  - Client requests a ZK proof from the Aptos Decentralized Prover Network, providing the JWT and EPK.
  - Prover Network computes and returns the ZK proof to the client.
  - Client sends a transaction to the SENA L2 sequencer, including the EPK and ZK proof.
  - SENA L2 sequencer's Aptos Signature Verification Module verifies the ZK proof.
  - If valid, the transaction is processed, and the L2 address is derived and associated with the user's session.

**Fraud proof interaction.** Proof verification runs inside the STF, so an incorrectly accepted ZK proof is itself a state transition fault and is challengeable like any other. This places a practical constraint on the verifier implementation: it must be expressible in the one-step execution target, and its instruction count must be bounded, since a disputed verification must ultimately bisect down to a single adjudicable instruction. Verification routines that are cheap on native hardware but pathological under step-wise emulation are to be avoided.

### 3.2 Social Connect Identity Index

- **Design Principles:** Privacy-preserving, fast lookup, multi-mapping support.
- **Data Structure:** A key-value store within RocksDB, where keys are cryptographic hashes of human-readable identifiers, and values are SENA L2 addresses. A separate index maps SENA L2 addresses to multiple hashed social identifiers.
- **Hashing Algorithm:** SHA-256 with a network-wide salt to prevent rainbow table attacks.
- **API:** RPC endpoint for `resolve_address(hashed_identifier)` and `get_social_links(sena_address)`.

### 3.3 Multi-Asset Gas Paymaster Engine

- **Design Principles:** Flexibility, user convenience, economic stability.
- **Components:**
  - **Whitelisted Stablecoin Ledger:** A state variable within the Sovereign SDK runtime that stores the list of approved stablecoins and their current exchange rates to APT.
  - **Gas Calculation Logic:** Determines the gas cost of a transaction in terms of APT, then converts it to the equivalent amount in the chosen stablecoin.
  - **Fee Extraction Logic:** Deducts the stablecoin fee directly from the user's transaction payload or account balance.
  - **System-Controlled Vault:** A dedicated L2 account that collects stablecoin fees.
  - **Aptos L1 Bridge Integration:** Logic within the sequencer to interact with the Aptos L1 Move contract to convert collected stablecoins to APT and fund L1 state commitments.
- **Flow:**
  - User submits a transaction, specifying a preferred whitelisted stablecoin for gas.
  - Sequencer calculates gas cost in APT.
  - Sequencer converts APT cost to stablecoin equivalent using current exchange rates.
  - Sequencer extracts stablecoin fee from user.
  - Stablecoin fees are deposited into the System-Controlled Vault.
  - Periodically, the sequencer initiates a cross-chain transaction via the Aptos L1 bridge to convert a portion of the vault's stablecoins to APT to cover L1 state commitment costs.

**Fraud proof interaction.** Exchange rates must enter the state through ordered, replayable transactions rather than being read from an ambient source at execution time (REQ-GAS-010). If the sequencer could consult a live price feed mid-execution, a verifier replaying the same batch later would compute different fees, diverge from an honest assertion, and open a challenge it would then lose. Rate updates are therefore modelled as ordinary state-transitioning transactions within the batch.

### 3.4 Optimistic L2 Governance Layer

- **Design Principles:** Decentralization, efficiency, low overhead.
- **Components:**
  - **Staking Module:** Manages \$SENA token staking for voting participation.
  - **Off-Chain Voting Platform:** (External to L2 node) Where token holders cast votes.
  - **Governance Multi-Signature Council:** A pre-defined set of trusted entities responsible for signing valid off-chain vote results.
  - **Administrative Module (L2 Node):** Ingests signed payloads, verifies signatures, and applies

state updates.

- **Flow:**

- Token holders stake \$SENA and vote on parameter changes on an off-chain platform.
- If a vote passes, the result (e.g., new stablecoin whitelisted) is formatted into a payload.
- The Governance Multi-Signature Council signs the payload.
- The signed payload is submitted to the SENA L2 sequencer.
- The Administrative Module verifies the council's signatures.
- Upon successful verification, the system variables in the Sovereign SDK runtime are updated.

**Fraud proof interaction.** The council's authority is bounded by construction. Governance payloads are applied as ordinary transactions inside a batch, so a council signature that authorises an invalid state change produces an invalid assertion and is challengeable exactly like sequencer fraud. Correspondingly, `sena::assertions` and `sena::disputes` expose no governance-callable entry point capable of finalizing a pending assertion, dismissing a challenge, or setting `CHALLENGE_WINDOW` below its hard floor (REQ-GOV-007). Parameters governance *can* reach — bond sizes, timeouts above their floors, the proposer allowlist — are held in a separate `sena::params` region that the dispute path reads but that cannot reach back into a dispute in progress.

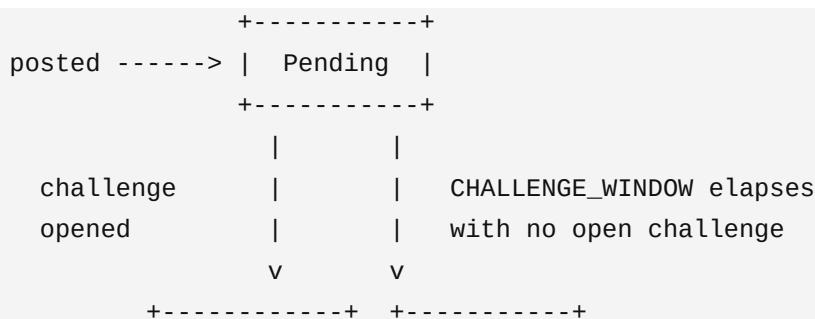
### 3.5 Fraud Proof and Dispute Resolution System

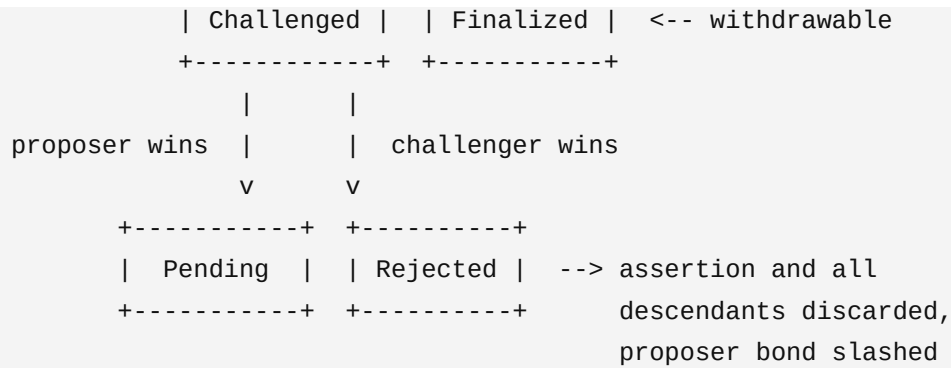
- **Design Principles:** Permissionless verification, no trusted third party in the resolution path, bounded on-L1 cost, and an honest party that always wins.

#### 3.5.1 The Assertion Chain

Assertions form a linked chain on L1, each referencing its parent by hash and carrying the pre-state root, post-state root, and batch commitment. An assertion is valid only if its pre-state root matches its parent's post-state root, which means a successful challenge invalidates not only the disputed assertion but every descendant built on top of it.

State machine for a single assertion:





Multiple challenges may be open against the same assertion concurrently; the assertion finalizes only when all of them have resolved in the proposer's favour and the window has elapsed (REQ-FRAUD-006).

### 3.5.2 Data Availability

Fraud proofs are worthless without the data to detect fraud in the first place. A sequencer that commits a state root while withholding the transactions that supposedly produced it cannot be challenged, because no verifier can compute the correct answer to compare against. SENA therefore treats unavailable data as invalid data (REQ-FRAUD-010).

Batch data is published to Aptos L1 alongside the assertion, inheriting L1 availability guarantees directly. Where batch size makes this uneconomic, the design permits publication to an approved DA layer with the commitment recorded in the assertion; in that configuration, the DA layer's availability guarantee becomes part of SENA's trust assumptions and must be disclosed as such.

A DA challenge is the cheap, non-interactive path: a challenger asserts the data is unavailable, and the proposer's only rebuttal is to publish it on L1 within DA\_RESPONSE\_TIMEOUT. Failure to do so rejects the assertion without any execution being adjudicated.

### 3.5.3 The Bisection Protocol

When a verifier's computed post-state root disagrees with an assertion, both parties agree on the starting state and disagree on the ending state. It follows that there is a first step at which they diverge. Bisection finds it, in  $\log_2(N)$  rounds for a trace of  $N$  steps, without either party having to prove anything about the other steps.

1. The challenger opens a dispute, posting CHALLENGER\_BOND and its own claimed post-state root.
2. The defender (the proposer) publishes  $k$  intermediate state commitments dividing the trace into  $k+1$  segments.
3. The challenger names the first segment whose end commitment it disagrees with.
4. Both parties recurse into that segment, repeating from step 2.

5. When a segment narrows to a single step, the parties have an agreed pre-step state and a disputed post-step state. Bisection ends and one-step proving begins.

A k-ary dissection rather than strict binary reduces round count — and therefore wall-clock dispute duration and total L1 gas — at the cost of larger individual moves. The parameter is tuned against Aptos L1 transaction size limits during implementation.

Each move is bounded by `MOVE_TIMEOUT`, and each party additionally draws from a per-dispute chess clock (`REQ-FRAUD-014`, `REQ-FRAUD-015`). The clock, rather than a per-move timeout alone, is what prevents an adversary from stretching a dispute toward the challenge window by always responding at the last possible moment. Clock budgets are sized with headroom for L1 congestion, because an honest party defeated by a full mempool is a security failure, not a fair loss.

### 3.5.4 The One-Step Proof

The final step is adjudicated by Aptos L1 itself. This requires the L1 to execute a single instruction of the SENA STF, which in turn requires the STF to be compiled to an instruction set simple enough to interpret inside a Move contract.

**Execution target.** The STF is compiled to RV32IM, a deterministic, fixed, well-specified RISC-V subset with no floating point. This choice is close to free: Sovereign SDK's zkVM integration already targets RISC-V, so the same reproducible ELF serves both the zk path and the fraud proof path, and there is one set of execution semantics to audit rather than two. The binary's hash is recorded on L1 and referenced by every assertion (`REQ-FRAUD-033`), so a dispute is always adjudicated against the exact code that produced the assertion.

**Machine state commitment.** A step's machine state is committed as a Merkle root over the register file, program counter, and memory pages, composed with the NOMT root of the L2 state the step may touch. Committing the machine rather than only the L2 state is what makes a single instruction independently checkable.

**Proof contents.** The challenger submits the instruction word, the pre-step machine commitment, the claimed post-step commitment, and Merkle inclusion proofs for every register and memory page the instruction reads or writes — together with NOMT inclusion proofs where the step touches L2 state.

**Adjudication.** `sena : : osp` verifies the inclusion proofs against the agreed pre-step commitment, decodes and executes the single instruction, recomputes the post-step commitment, and compares. One instruction, a handful of Merkle paths: the whole procedure fits comfortably within an Aptos L1 transaction (`NFR-PERF-007`).

### 3.5.5 Resolution and Rollback

A successful challenge rejects the assertion and every descendant, rolls the canonical state back to the last finalized assertion, slashes the proposer's bond, and removes the proposer from the permitted set pending governance reinstatement (REQ-FRAUD-019 to REQ-FRAUD-022).

Bond distribution deliberately does not pay the full slashed amount to the winner. A challenger who receives everything the proposer loses has an incentive to collude with — or simply be — the proposer, posting deliberately invalid assertions to harvest bonds in a wash. The winner receives a reward calibrated to exceed verification and L1 gas costs by a healthy margin; the remainder goes to the treasury.

Rollback is a genuine operational event, not a theoretical one: reverted L2 blocks contained transactions users believed confirmed. Recovery re-derives state from the last finalized assertion plus published batch data, and the set of reverted blocks is exposed through the RPC API (REQ-FRAUD-023) so that applications, exchanges, and indexers can reconcile rather than silently serve stale history.

### 3.5.6 Liveness

A fraud proof system that a sequencer can neutralise by refusing service is not a security guarantee. Two mechanisms address this. The forced-inclusion inbox on `sena::bridge` lets any user submit a transaction directly to L1; if the sequencer does not include it within `FORCED_INCLUSION_TIMEOUT`, it becomes includable by anyone and the sequencer's bond is subject to slashing (REQ-FRAUD-028, REQ-FRAUD-029). And if the sequencer stops producing assertions entirely, any bonded party may propose the next one after `PROPOSER_LIVENESS_TIMEOUT` (REQ-FRAUD-030), so the chain can finalize and users can exit without the sequencer's cooperation.

### 3.5.7 Parameters

| Parameter          | Default                         | Bound                                                            | Governance-adjustable |
|--------------------|---------------------------------|------------------------------------------------------------------|-----------------------|
| CHALLENGE_WINDOW   | 7 days                          | Hard floor 24 h,<br>enforced in<br><code>sena::assertions</code> | Yes, above floor      |
| SEQUENCER_BOND_MIN | Set at launch per NFR-SEC-009   | —                                                                | Yes                   |
| CHALLENGER_BOND    | Sized to deter griefing, not to | —                                                                | Yes                   |

| Parameter                 | Default                       | Bound                         | Governance-adjustable |
|---------------------------|-------------------------------|-------------------------------|-----------------------|
|                           | exclude ordinary participants |                               |                       |
| MOVE_TIMEOUT              | 6 h                           | —                             | Yes                   |
| DISPUTE_CLOCK_BUDGET      | 48 h per party                | Must be <<br>CHALLENGE_WINDOW | Yes                   |
| DA_RESPONSE_TIMEOUT       | 12 h                          | —                             | Yes                   |
| FORCED_INCLUSION_TIMEOUT  | 24 h                          | —                             | Yes                   |
| PROPOSER_LIVENESS_TIMEOUT | 24 h                          | —                             | Yes                   |
| BISECTION_ARITY (k)       | Tuned to L1 tx size limits    | —                             | Yes                   |

## 4. Data Design

### 4.1 Data Structures

#### Account State:

```
struct Account {
    address: L2Address,
    balance: HashMap<AssetId, u128>,
    // ... other account-related data
}
```

#### Social Link Mapping:

```
struct SocialLink {
    hashed_identifier: HashedIdentifier,
    l2_address: L2Address,
    // ... metadata like provider, timestamp
}
```

### Whitelisted Assets:

```
struct WhitelistedAsset {
    asset_id: AssetId,
    symbol: String,
    exchange_rate_to_apt: u128, // Fixed-point representation
}
```

### Governance Parameters:

```
struct GovernanceParameter {
    key: String,
    value: String, // Or specific type based on parameter
    last_updated_block: u64,
}
```

### Assertion:

```
struct Assertion {
    id: AssertionId,
    parent: AssertionId,
    proposer: L1Address,
    block_range: (u64, u64),
    pre_state_root: StateRootHash,
    post_state_root: StateRootHash,
    batch_commitment: DaCommitment,
    tx_count: u64,
    stf_binary_hash: [u8; 32],
    bond: u128,
    posted_at_l1: u64, // L1 timestamp; challenge window runs from here
    status: AssertionStatus, // Pending | Challenged | Finalized | Rejected
    open_challenges: u32,
}
```

### Challenge:

```
struct Challenge {
    id: ChallengeId,
    assertion: AssertionId,
    challenger: L1Address,
    bond: u128,
    claimed_post_state_root: StateRootHash,
    stage: DisputeStage, // Bisecting | AwaitingOsp | Resolved
}
```

```

    segment: (u64, u64),      // current disputed trace interval [lo, hi)
    turn: Party,              // Defender | Challenger
    move_deadline: u64,
    clock_remaining: (u64, u64), // (defender, challenger) seconds
}

```

### **Bisection Move:**

```

struct BisectionMove {
    challenge: ChallengeId,
    segment: (u64, u64),
    commitments: Vec<MachineCommitment>, // k intermediate states
    submitted_by: Party,
    l1_block: u64,
}

```

### **One-Step Proof:**

```

struct OneStepProof {
    challenge: ChallengeId,
    step_index: u64,
    instruction: u32,                // RV32IM instruction word
    pre_machine: MachineCommitment,
    post_machine: MachineCommitment,
    register_proofs: Vec<MerkleProof>,
    memory_page_proofs: Vec<MerkleProof>,
    state_proofs: Vec<NomtInclusionProof>, // L2 state read/written by the step
}

```

### **Machine Commitment:**

```

struct MachineCommitment {
    pc: u64,
    registers_root: [u8; 32],
    memory_root: [u8; 32],
    state_root: StateRootHash, // NOMT root of L2 state at this step
}

```

## **4.2 Database Schema (RocksDB)**

RocksDB will be used as a key-value store. Column families will be utilized to logically separate different types of data, improving query performance and organization.

| Column Family Name      | Key Type                 | Value Type                 | Description                                                                                                                                            |
|-------------------------|--------------------------|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| accounts                | L2Address                | Account (serialized)       | Stores individual user account states.                                                                                                                 |
| social_links_by_hash    | HashedIdentifier         | L2Address                  | Maps hashed social IDs to L2 addresses.                                                                                                                |
| social_links_by_address | L2Address                | Vec\<br><HashedIdentifier> | Maps L2 addresses to multiple hashed social IDs.                                                                                                       |
| whitelisted_assets      | AssetId                  | WhitelistedAsset           | Stores details of approved stablecoins.                                                                                                                |
| governance_params       | String (param name)      | GovernanceParameter        | Stores network configuration parameters.                                                                                                               |
| state_roots             | BlockNumber              | StateRootHash              | Records L2 state roots committed to Aptos L1.                                                                                                          |
| transactions            | TxHash                   | Transaction                | Stores historical L2 transactions.                                                                                                                     |
| assertions              | AssertionId              | Assertion                  | Local mirror of the L1 assertion chain and its status.                                                                                                 |
| batch_data              | AssertionId              | Vec\<br><Transaction>      | Ordered batch data as published, retained for the challenge window so the node can re-execute or defend.                                               |
| challenges              | ChallengeId              | Challenge                  | Open and recently resolved disputes the node is party to or observing.                                                                                 |
| exec_checkpoints        | (AssertionId, StepIndex) | MachineCommitment          | Periodic machine-state snapshots within a batch's trace, so any step can be reached by replaying from the nearest checkpoint instead of from the batch |

| Column Family Name | Key Type    | Value Type    | Description                                                                                                             |
|--------------------|-------------|---------------|-------------------------------------------------------------------------------------------------------------------------|
|                    |             |               | start.                                                                                                                  |
| verifier_results   | AssertionId | StateRootHash | The post-state root this node computed independently, compared against the asserted one to decide whether to challenge. |

Checkpoint density in `exec_checkpoints` trades storage against the time to produce a bisection move. Density is chosen so that replaying from the nearest checkpoint completes well inside `MOVE_TIMEOUT` on the reference verifier hardware of REQ-FRAUD-026.

### 4.3 Merkle Tree Implementation (NOMT)

NOMT will be integrated with RocksDB to provide a merklized state. This allows for efficient generation of state proofs, crucial for L1 commitments and user fund withdrawals. The NOMT will maintain a Merkle tree over the entire L2 state, enabling quick verification of state changes without recomputing the entire tree.

NOMT serves the fraud proof path as well: the inclusion proofs carried by a One-Step Proof are NOMT proofs, and the `sena::osp` contract contains a Move implementation of NOMT proof verification. This is a hard compatibility constraint — the Move verifier and the Rust NOMT implementation must agree exactly on hashing, node encoding, and tree layout, and a divergence between them is a critical security bug.

## 5. Security Design

### 5.1 Inherited Security from Aptos L1

- **State Root Commitment:** The L2 sequencer will periodically compute a cryptographic hash of its entire state (the State Root) and submit it to a designated Move contract on the Aptos L1, as a bonded assertion. This process leverages Aptos's robust consensus mechanism to ensure the immutability and finality of the L2 state history.
- **Emergency Withdrawal Mechanism:** The L1 Move contract will include functionality allowing users to initiate withdrawals of their L2 funds directly from L1, using a valid L2 state proof rooted in a finalized assertion. This mechanism acts as a safety net, guaranteeing fund recoverability even if the L2 sequencer is compromised or offline.

## 5.2 Keyless JWT Validation Architecture

- **Ephemeral Key Pairs:** Client applications will generate short-lived ephemeral key pairs for each user session, enhancing security by limiting the exposure window of any single key.
- **Decentralized ZKP Generation:** The computationally intensive task of generating ZK proofs will be offloaded to the Aptos Decentralized Prover Network. This prevents the L2 sequencer from becoming a bottleneck or a target for denial-of-service attacks related to ZKP computation.
- **On-Chain ZKP Verification (Rust):** The SENA L2 node will incorporate Aptos's battle-tested, open-source Rust libraries for ZKP signature verification. This verification is lightweight and performed efficiently during transaction processing, ensuring the authenticity of keyless transactions without significant performance overhead.

## 5.3 Privacy-Preserving Identifiers

- **Hashed Social Identifiers:** All human-readable social identifiers will be stored as SHA-256 hashes, combined with a unique network salt. This prevents direct exposure of personal information in the public L2 state while still allowing for unique identification and mapping.
- **OIDC-based Address Derivation:** L2 addresses will be deterministically derived from OIDC claims (\$aud, \$sub) and a pepper value, ensuring that user identities are linked to their blockchain addresses in a secure and privacy-preserving manner.

## 5.4 Fraud Proof Security Analysis

**What the system defends against.** A sequencer that is compromised, coerced, or simply buggy, and that commits a state root not derivable from the published transactions — inflating a balance, reassigning a social link, spending from an account without authorisation, or corrupting the whitelist. Any such assertion diverges from what an honest verifier computes, and the verifier wins the resulting dispute because it holds the correct execution trace and the adversary does not.

**The core argument.** Both parties agree on the pre-state and disagree on the post-state, so they diverge at some first step. Bisection locates that step in logarithmic rounds. At that step they share an agreed input and disagree on the output of a single instruction — and Aptos L1 can settle that question by executing the instruction itself. An honest party is never forced into a false claim at any round, so it never loses a bisection it entered correctly.

**Threats and mitigations:**

| Threat                                                                   | Mitigation                                                                                 |
|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Sequencer withholds batch data so fraud cannot be detected               | DA challenge; unavailable data is treated as invalid (§3.5.2)                              |
| Sequencer stalls the dispute until the challenge window lapses           | Per-move timeouts plus per-party chess clocks bounded below the window (§3.5.3)            |
| Adversary floods spurious challenges to delay finalization               | Challenger bonds slashed on loss; delay per unit of bond is bounded (NFR-SEC-011)          |
| Adversary censors challengers on L1                                      | Dispute duration is a small fraction of the window; clock budgets assume congestion        |
| Sequencer and a sham challenger collude to wash bonds                    | Winner receives a calibrated reward, not the full slashed bond (§3.5.5)                    |
| Nondeterminism makes an honest verifier diverge from an honest sequencer | Strict determinism requirements and RV32IM target (REQ-FRAUD-031, §3.5.4)                  |
| Governance is used to rescue a fraudulent assertion                      | Dispute contracts expose no governance entry point; window floor enforced in code (§3.4)   |
| Verification is too expensive, so nobody verifies                        | Commodity-hardware requirement and reimbursed challenge costs (REQ-FRAUD-026, NFR-SEC-010) |

**What the system does not defend against, stated plainly:**

- **No honest verifier.** If every party capable of verifying is absent, compromised, or bribed into silence for the whole window, fraud finalizes. The protocol reduces trust from "the sequencer is honest" to "someone is watching" — a far weaker assumption, but not no assumption. Funding independent watchtowers is an operational necessity, not an optional extra.
- **Sustained L1 censorship.** If Aptos L1 excludes challenge transactions for the entire challenge window, no challenge can land. This is inherited from the L1 and is a principal reason the window is measured in days rather than hours.

- **A bug in the one-step verifier.** `sena::osp` is the final arbiter; if its RV32IM semantics or NOMT proof verification diverge from the Rust implementation, it can decide a dispute wrongly in either direction. This is the single highest-value target in the system and is why NFR-SEC-012 places it under independent audit and top-tier bounty.
- **Sequencer censorship and MEV.** Fraud proofs constrain *what state the sequencer can assert*, not *which transactions it chooses to include or in what order*. Forced inclusion bounds censorship; ordering discretion remains with the single sequencer until sequencing is decentralised.
- **The withdrawal delay.** Trust-minimised exit takes a challenge window. Third-party fast-withdrawal liquidity can mask it for ordinary amounts, but those providers are themselves a trust assumption for the users who rely on them.

## 6. Interface Design

### 6.1 External Interfaces

- **RPC API:** Standard JSON-RPC interface for client applications to submit transactions, query state, and interact with L2 modules. The API additionally exposes assertion and dispute state: `sena_getAssertion`, `sena_getAssertionStatus`, `sena_getFinalizedHeight`, `sena_getOpenChallenges`, and `sena_getRevertedBlocks`. Transaction receipts carry both a soft-confirmation status and the identifier of the assertion covering them, together with that assertion's finalization state, so that applications can distinguish the two per REQ-CORE-005.
- **Aptos L1 Bridge Interface:** A Rust module within the sequencer that interacts with the Aptos L1 Move contract for state root commitments and asset transfers.
- **Dispute Interface:** The entry points on `sena::assertions` and `sena::disputes` through which any party posts assertions, opens challenges, submits bisection moves, and submits One-Step Proofs. Permissionless by design (REQ-FRAUD-011).
- **Client-Side SDKs:** Libraries (e.g., JavaScript, Mobile) for developers to easily integrate with SENA's keyless authentication and other features. SDKs surface finalization state as a first-class property of a transaction rather than an advanced query, so that applications handling significant value are not silently treating soft confirmation as settlement.

### 6.2 Internal Interfaces

- **Sovereign SDK Module Interfaces:** Well-defined Rust traits and structs for inter-module communication within the Sovereign SDK runtime.
- **RocksDB Interface:** Rust bindings for interacting with the RocksDB key-value store.
- **Dispute Engine Interface:** The trait through which the node drives a dispute in either role, allowing the

same logic to defend an assertion as sequencer or prosecute one as challenger.

- **Execution Trace Interface:** The interface by which the Dispute Engine replays a batch step-wise, reaches an arbitrary step from the nearest checkpoint, and emits machine commitments at requested offsets.

## 7. Future Considerations

- **Decentralized Sequencer:** While initially a single-sequencer model, future iterations may explore decentralized sequencer designs for enhanced censorship resistance and fault tolerance. Fraud proofs constrain sequencer *correctness* but not ordering discretion; decentralised sequencing addresses the remainder.
- **Permissionless Proposers:** Extending assertion proposal beyond the sequencer to any bonded party in the normal case, not only after a liveness timeout.
- **ZK One-Step Resolution:** Replacing the interactive bisection game with a single validity proof of the disputed batch. Because the STF already compiles to RISC-V for the zkVM, this is an evolution of the same binary rather than a rewrite, and it would collapse the challenge window from days to proof-generation time.
- **Shortened Challenge Windows:** Reducing CHALLENGE\_WINDOW as verifier participation, tooling maturity, and L1 censorship resistance justify it — never below the floor enforced in `sena::assertions`.
- **Additional L1 Integrations:** Potential integration with other L1 blockchains beyond Aptos.

## References

[1] Software Requirements Specification (SRS) for SENA Network. [2] SENA Network: Layer 2 Modular Infrastructure Documentation. [3] Sovereign SDK Documentation. Sovereign Labs. Available at: <https://docs.sovereign.xyz/> [4] RocksDB. GitHub. Available at: <https://github.com/facebook/rocksdb> [5] Aptos Keyless Accounts. Aptos Docs. Available at: <https://aptos.dev/concepts/accounts/#keyless-accounts> [6] NOMT: Nearly-Optimal Merkle Trie. GitHub. Available at: <https://github.com/aptos-labs/aptos-core/tree/main/storage/aptdb/src/schema/nomt> [7] AptosDB: The Aptos Database. Aptos Docs. Available at: <https://aptos.dev/nodes/aptos-node-references/aptdb/> [8] Merkle Trees. Wikipedia. Available at: [https://en.wikipedia.org/wiki/Merkle\\_tree](https://en.wikipedia.org/wiki/Merkle_tree) [9] Zero-Knowledge Proofs. Wikipedia. Available at: [https://en.wikipedia.org/wiki/Zero-knowledge\\_proof](https://en.wikipedia.org/wiki/Zero-knowledge_proof) [10] OIDC and ZKP in Aptos. Aptos Labs Blog. Available at: <https://aptoslabs.com/blog/aptos-keyless-accounts-oidc-zkp> [11] RISC-V Instruction Set Manual, Volume I: Unprivileged ISA. RISC-V International. Available at: <https://riscv.org/technical/specifications/>